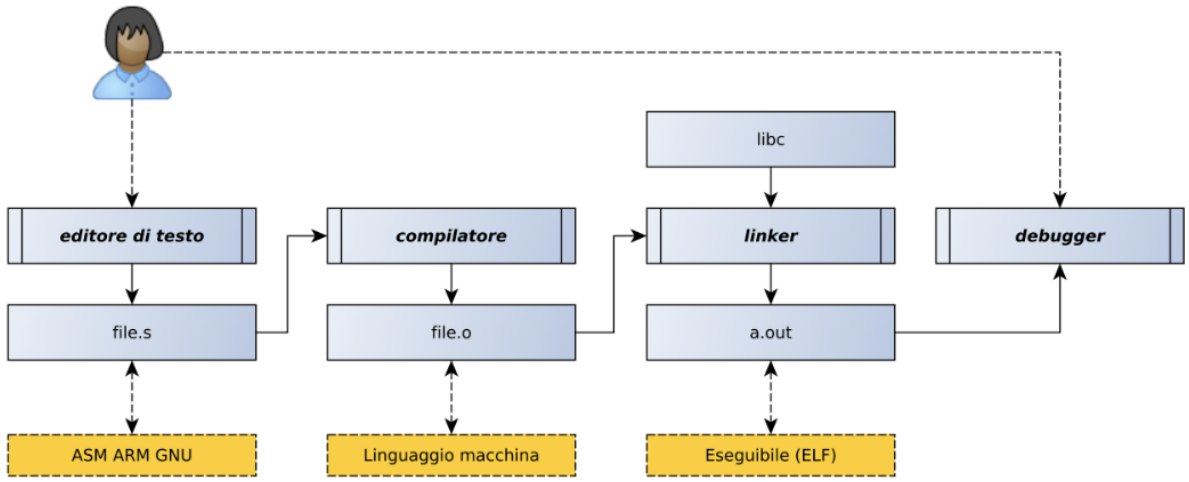
Assembly

Il programma viene scritto in un file .s (sorgente).
Bisogna rispettare la sintassi GNU per le diverse architetture



Workflow per la realizzazione di un programma Assembler in ARM tool GNU

Etichette

```
main:
```

le etichette sono seguite dal carattere ":"
(in sintassi ARM non vanno messi i due punti)

Spesso vengono utilizzate per marcare posizioni di memoria o punti di destinazione per istruzioni di salto condizionate o incondizionate.

```
_start
```

L'etichetta `_start` è globale, ovvero che può esser vista anche in altri file sorgente

Le etichette possono essere utilizzate con istruzioni di salto come `b` (branch) o `bl` (branch with link). Ad esempio, `b end_program` salterà all'etichetta `end_program`.

> 💡 È importante notare che le etichette sono usate per scopi di riferimento durante l'assemblaggio e non si traducono direttamente in istruzioni macchina.
> Sono solo punti di riferimento nel tuo codice sorgente.

Istruzioni

- ADDIZIONE

```
ADD a, b, c -> a = b + c
```

- SOTTRAZIONE

```
SUB a, b, c -> a = b - c
```

```
ADD temp, b, c -> temp = b + c
SUB a, temp, d -> a = temp - d
```

```
; questo è un commento
```

Ciao sdasdk

Registri

Gli **operandi** possono essere memorizzati nei registri del processore, o in memoria, oppure essere costanti memorizzate nelle istruzioni stesse.
I calcolatori usano locazioni diverse per gli operandi al fine di ottimare la velocità e capienza.

Dato che richiede tempi lunghi recuperare gli operandi *salvati* in memoria, ARM mette a disposizione **16 registri**, ovvero dove salvare gli operandi che vengono utilizzati più spesso così da diminuire i tempi di ricerca.

```
R0 = a, R1 = b, R2 = c
ADD R0, R1, R2 -> a = b + c
```

Tabella 6.1 Registri di ARM.

Nome	Utilizzo
R0	Parametro/valore da restituire/variabile temporanea
R1-R3	Parametri/variabili temporanee
R4-R11	Variabili salvate
R12	Variabile temporanea
R13 (SP)	*Stack Pointer*
R14 (LR)	*Link Register*
R15 (PC)	*Program Counter*

```
MOV R4, #0 -> R4 = 0
```

L'istruzione `MOV` serve per inizializzare i valore nei registri o per usare i registri come operandi sorgente, ad esempio:

```
MOV R1, R7 -> copia il contenuto di R7 in R1
```

Memoria

Rispetto ai registri è più grande ma più lenta.

LoaD Register

Carica un valore dalla memoria principale in un registro.

La sintassi generale dell'istruzione `LDR` è la seguente:

```
LDR Rd, [Rn, #offset]
```

Dove:

- `Rd` è il registro di destinazione in cui verrà caricato il valore.
- `Rn` è il registro che contiene l'indirizzo base di memoria.
- `#offset` è un valore immediato (costante) o un registro che indica l'offset dall'indirizzo base di memoria.

Dopo che viene eseguito, `Rd` contiene il dato memorizzato all'indirizzo `#offset`

STore Register

Memorizza il contenuto di un registro nella memoria principale.

La sintassi generale dell'istruzione `STR` è la seguente:

```
STR Rd, [Rn, #offset]
```

Dove:

- `Rd` è il registro sorgente contenente il valore da memorizzare.
- `Rn` è il registro che contiene l'indirizzo base di memoria.
- `#offset` è un valore immediato (costante) o un registro che indica l'offset dall'indirizzo base di memoria.

Ecco un esempio:

```
STR r0, [r1, #4]
```

In questo esempio, il contenuto del registro `r0` viene memorizzato nella memoria all'indirizzo ottenuto sommando 4 all'indirizzo contenuto in `r1`. Quindi, il valore di `r0` viene scritto nella memoria all'indirizzo specificato.

MoVe and Not

MVN esegue la negazione bit a bit della seconda sorgente (la prima sorgente non esiste) e scrive il risultato nel registro destinazione.

BIC

L'istruzione `bic` è molto utile per "mascherare" bit (cioè per forzare a 0 i bit che non interessano). `BIC R6, R1, R2` calcola `R1 AND NOT R2`, cioè azzera i bit che sono a 1 in R2.

Istruzioni di traslazione

In ARM, le istruzioni di traslazione sono utilizzate per spostare i bit all'interno di un registro da sinistra a destra o da destra a sinistra. Le istruzioni di traslazione possono essere divise in due categorie principali: traslazioni aritmetiche e traslazioni logiche.

1. **Traslazioni Aritmetiche**:
   - `ASR` (Arithmetic Shift Right): Questa istruzione sposta i bit del registro a destra e inserisce il bit più significativo (MSB) in base al bit di segno.

     Esempio:

     ```
     ASR r1, r2, #2
     ; Shift right r2 di 2 posizioni aritmetiche, risultato
     ```

```
in r1
```

2. **Traslazioni Logiche**:

   - `LSL` (Logical Shift Left): Questa istruzione sposta i bit del registro a sinistra, inserendo zero nel bit meno significativo (LSB) e spostando il bit di carry nella flag di carry.

     Esempio:

     ```
     LSL r1, r2, #3
     ; Shift left r2 di 3 posizioni logiche, risultato in r1
     ```

   - `LSR` (Logical Shift Right): Questa istruzione sposta i bit del registro a destra, inserendo zero nel bit più significativo (MSB) e spostando il bit di carry nella flag di carry.

     Esempio:

     ```
     LSR r1, r2, #4
     ; Shift right r2 di 4 posizioni logiche, risultato in r
     1
     ```

   - `ROR` (Rotate Right): Questa istruzione sposta i bit del registro a destra, inserendo il bit meno significativo (LSB) nel bit più significativo (MSB) e spostando il bit di carry nella flag di carry.

     Esempio:

     ```
     ROR r1, r2, #5
     ; Rotate right r2 di 5 posizioni, risultato in r1
     ```

Queste istruzioni di traslazione sono utili per operazioni di manipolazione dei bit e possono essere usate in diverse situazioni a seconda delle esigenze dell'algoritmo o dell'applicazione.

Istruzioni di moltiplicazione

In ARM, esistono diverse istruzioni di moltiplicazione che consentono di eseguire operazioni di moltiplicazione su registri. Alcune delle istruzioni più comuni includono:

1. `MUL` **(Multiply)**: Esegue una moltiplicazione tra due registri e memorizza il risultato in un terzo registro.

   ```
   MUL Rd, Rn, Rm
   ; Rd = Rn * Rm
   ```

   Esempio:

   ```
   MUL r1, r2, r3
   ; r1 = r2 * r3
   ```

2. `MLA` **(Multiply and Accumulate)**: Esegue una moltiplicazione tra due registri, somma un terzo registro al risultato e memorizza il risultato in un quarto registro.

   ```
   MLA Rd, Rn, Rm, Ra
   ; Rd = (Rn * Rm) + Ra
   ```

   Esempio:

   ```
   MLA r1, r2, r3, r4
   ; r1 = (r2 * r3) + r4
   ```

3. `UMULL` **(Unsigned Multiply Long)** e `SMULL` **(Signed Multiply Long)**: Eseguono una moltiplicazione tra due registri, generando un risultato a 64 bit che viene memorizzato in due registri (uno per i 32 bit meno significativi e uno per i 32 bit più significativi).

   ```
   UMULL RdLo, RdHi, Rn, Rm   ; Unsigned Multiply Long
   SMULL RdLo, RdHi, Rn, Rm   ; Signed Multiply Long
   ```

   Esempio:

```
UMULL r1, r2, r3, r4   ; r1 = r3 * r4 (risultato in r2:r1)
```

4. `MLAL` **(Multiply Accumulate Long)**: Esegue una moltiplicazione tra due registri, somma il risultato a un valore a 64 bit rappresentato da due registri e memorizza il risultato a 64 bit in altri due registri.

   ```
   MLA RdLo, RdHi, Rn, Rm, RaLo, RaHi   ; Rd = (Rn * Rm) + (R
   aLo:RaHi)
   ```

   Esempio:

   ```
   MLA r1, r2, r3, r4, r5, r6   ; r1 = (r3 * r4) + (r5:r6)
   ```

Queste istruzioni di moltiplicazione sono utilizzate per eseguire operazioni di moltiplicazione e accumulo in vari contesti, sia per numeri interi sia per numeri in virgola mobile. La scelta della specifica istruzione dipenderà dalle esigenze dell'applicazione e dal tipo di operazioni che si desidera eseguire.

Flag di condizione

I flag di condizione, o semplicemente flag, sono bit speciali presenti nel registro del flag di uno stato del processore.
Questi bit vengono impostati o cancellati in risposta alle istruzioni eseguite dal processore.

I flag di condizione sono utilizzati per mantenere informazioni sul risultato di un'operazione e guidano il flusso di controllo attraverso l'esecuzione del programma.

Nei processori ARM, i flag di condizione comuni includono:

1. **Zero Flag (Z)**: Viene impostato se il risultato di un'operazione è zero.
2. **Negative Flag (N)**: Viene impostato se il risultato di un'operazione è negativo.

3. **Carry Flag (C)**: Viene impostato se c'è un riporto da un'operazione aritmetica o se il bit di carry è coinvolto.
4. **Overflow Flag (V)**: Viene impostato se si verifica un overflow aritmetico, ad esempio, se il risultato di un'operazione supera la rappresentazione numerica massima o minima.
5. **Flag di Link (o Link Register, LR)**: Viene utilizzato per salvare l'indirizzo di ritorno quando si effettua una chiamata a una subroutine. Spesso indicato come `L` o `Link` nelle istruzioni.
6. **Flag di Half-Carry (H)**: Utilizzato in alcune operazioni aritmetiche per indicare un riporto dalla parte bassa di un byte alla parte alta di un byte in una cifra binaria.
7. **Overflow Overflow (Q)**: Usato in alcune operazioni di moltiplicazione per segnalare un overflow.
8. **Saturation Flag (Q)**: Viene utilizzato in alcune operazioni di saturazione per indicare se si è verificato un overflow o un underflow.
9. **Sticky Overflow Flag (Q)**: Usato in alcune operazioni di saturazione per indicare se un overflow o un underflow è mai avvenuto, anche se il risultato finale non è saturato.

L'accesso e l'utilizzo di questi flag possono variare a seconda dell'architettura specifica del processore ARM e delle istruzioni fornite.

Tabella 6.2 **Flag di condizione.**

Flag	Nome	Descrizione
N	*Negative*	Il risultato dell'istruzione è negativo, cioè il bit 31 vale 1
Z	*Zero*	Il risultato dell'istruzione è zero
C	*Carry*	L'istruzione ha generato riporto (*carry*) di uscita
V	*oVerflow*	L'istruzione ha causato traboccamento (*overflow*)

Mnemonici di condizione

Sono aggiunti a molte istruzioni assembly per specificare una condizione sotto la quale l'istruzione deve essere eseguita.
Queste istruzioni condizionali sono spesso utilizzate in combinazione con il salto condizionato (branch) o con altre istruzioni che possono essere eseguite solo in determinate condizioni.

Nei processori ARM gli mnemonici di condizione seguono l'istruzione e sono aggiunti per specificare la condizione sotto la quale l'istruzione deve essere eseguita.

Tabella 6.3 Mnemonici di condizione.

cond	Mnemonico	Nome	CondEse
0000	EQ	Uguale (*EQual*)	Z
0001	NE	Diverso (*Not Equal*)	$\bar{Z}$
0010	CS/HS	Attiva riporto/maggiore o uguale senza segno (*Carry Set/unsigned Higher or Same*)	C
0011	CC/LO	Disattiva riporto/minore senza segno (*Carry Clear/unsigned LOwer*)	$\bar{C}$
0100	MI	Meno/negativo (*MInus/negative*)	N
0101	PL	Più/positivo o nullo (*PLus/positive or zero*)	$\bar{N}$
0110	VS	Traboccamento/attiva traboccamento (*overflow/oVerflow Set*)	V
0111	VC	No traboccamento/disattiva traboccamento (*overflow/oVerflow Clear*)	$\bar{V}$
1000	HI	Maggiore senza segno (*unsigned HIgher*)	$\bar{Z}C$
1001	LS	Minore o uguale senza segno (*unsigned Lower or Same*)	Z OR $\bar{C}$
1010	GE	Maggiore o uguale con segno (*signed Greater than or Equal*)	$\overline{N \oplus V}$
1011	LT	Minore con segno (*signed Less Than*)	$N \oplus V$
1100	GT	Maggiore con segno (*signed Greater Than*)	$\bar{Z}$ $(\overline{N \oplus V})$
1101	LE	Minore o uguale con segno (*signed Less than or Equal*)	Z OR $(N \oplus V)$
1110	AL (o niente)	Sempre/incondizionato (*ALways/unconditional*)	Ignorato

Istruzioni di salto

Le istruzioni di salto, o branch instructions, in linguaggio assembly ARM sono utilizzate per modificare il flusso di esecuzione del programma. Queste istruzioni consentono di saltare a diversi punti nel codice in base a condizioni specifiche o senza alcuna condizione. Alcune delle istruzioni di salto più comuni in ARM includono:

1. `B` **(Branch)**: Esegue sempre il salto incondizionato all'indirizzo specificato.

```
B label   ; Salta incondizionatamente all'indirizzo della label
```

2. `BL` **(Branch with Link)**: Esegue un salto incondizionato all'indirizzo specificato e salva l'indirizzo successivo (indirizzo di ritorno) nel registro del link (LR).

```
BL label   ; Salta incondizionatamente all'indirizzo della label e salva l'indirizzo di ritorno in LR
```

3. `BEQ` **(Branch if Equal)**: Salta all'indirizzo specificato solo se il flag di zero è impostato (l'ultima operazione ha prodotto zero).

```
BEQ label   ; Salta se il flag di zero è impostato
```

4. `BNE` **(Branch if Not Equal)**: Salta all'indirizzo specificato solo se il flag di zero non è impostato.

```
BNE label   ; Salta se il flag di zero non è impostato
```

5. `BGT` **(Branch if Greater Than)**: Salta all'indirizzo specificato solo se il risultato dell'ultima operazione è stato maggiore di zero.

```
BGT label   ; Salta se il risultato è maggiore di zero
```

6. `BLT` **(Branch if Less Than)**: Salta all'indirizzo specificato solo se il risultato dell'ultima operazione è stato minore di zero.

```
BLT label   ; Salta se il risultato è minore di zero
```

Queste sono solo alcune delle istruzioni di salto più comuni in ARM. Le istruzioni di salto sono fondamentali per la creazione di strutture di controllo di flusso, come cicli (`for`, `while`) e strutture condizionali (`if`, `else`).

Le istruzioni di salto in assembly ARM possono essere divise in due categorie principali: salto condizionato e salto non condizionato.

1. **Salto Non Condizionato**:
   - **Istruzione Principale**: `B` (Branch) e `BL` (Branch with Link).
   - **Descrizione**: Queste istruzioni eseguono un salto incondizionato all'indirizzo specificato. Il salto avviene sempre, indipendentemente dai valori dei flag o dalle condizioni dell'ultimo risultato.
   - **Esempio**:

```
B label
; Salta incondizionatamente all'indirizzo della label

BL subroutine
; Salta incondizionatamente a subroutine e salva l'indi
rizzo di ritorno
```

2. **Salto Condizionato**:
   - **Istruzioni Principali**: `BEQ` (Branch if Equal), `BNE` (Branch if Not Equal), `BGT` (Branch if Greater Than), `BLT` (Branch if Less Than), ecc.
   - **Descrizione**: Queste istruzioni eseguono un salto all'indirizzo specificato solo se una determinata condizione è soddisfatta. La condizione è basata sui flag di stato del processore, come il flag di zero (Z), il flag di segno (N), o altri flag specifici dell'istruzione.

- **Esempio**:

```
BEQ label   ; Salta se il flag di zero è impostato
BNE label   ; Salta se il flag di zero non è impostato
BGT label   ; Salta se il risultato è maggiore di zero
```

💡 In breve, il salto non condizionato avviene sempre, indipendentemente dalle condizioni del programma. D'altra parte, il salto condizionato avviene solo se specifiche condizioni sono soddisfatte, in base ai flag di stato del processore.

Costrutti di selezione

I costrutti di selezione, comunemente noti come istruzioni condizionali, sono utilizzati per eseguire parti specifiche del codice in base al soddisfacimento di determinate condizioni.

In linguaggio assembly ARM, i costrutti di selezione possono essere implementati principalmente con istruzioni di salto condizionato.

Istruzione `BEQ` (Branch if Equal) - Selezione con "if"

```
; Codice eseguito se R0 non è uguale a R1
CMP R0, R1        ; Confronto tra R0 e R1
BEQ EqualLabel    ; Salta a EqualLabel se i registri sono ugu
ali

B   EndLabel      ; Salta a EndLabel

; Codice eseguito se R0 è uguale a R1
```

```
EqualLabel:

; Codice eseguito alla fine del costrutto di selezione
EndLabel:
```

ESEMPIO DI CODICE 6.13 **COSTRUTTO *IF***

Codice di alto livello	Codice *assembly* ARM

`if (mele == arance)`
`  f = i + 1;`
`f = f - i;`
	`; R0 = mele, R1 = arance, R2 = f, R3 = i`
	`  CMP R0, R1  ; mele == arance ?`
	`  BNE NOIF  ; se diverse, salta blocco if`
	`  ADD R2, R3, #1  ; blocco if: f = i + 1`
	`NOIF  SUB R2, R2, R3  ; f = f - i`

Istruzione `BNE` (Branch if Not Equal) - Selezione con "if-else"

```
; Codice eseguito se R0 è uguale a R1
CMP R0, R1        ; Confronto tra R0 e R1
BNE NotEqualLabel ; Salta a NotEqualLabel se i registri non s
ono uguali

B   EndLabel      ; Salta a EndLabel

; Codice eseguito se R0 non è uguale a R1
NotEqualLabel:

; Codice eseguito alla fine del costrutto di selezione
EndLabel:
```

ESEMPIO DI CODICE 6.14 **COSTRUTTO *IF/ELSE***

Codice di alto livello

```
if (mele == arance)
  f = i + 1;
else
  f = f - i;
```

Codice *assembly* ARM

```
; R0 = mele, R1 = arance, R2 = f, R3 = i
  CMP R0, R1              ; mele == arance ?
  BNE ELSE                ; se diverse, va a blocco ELSE
  ADD R2, R3, #1          ; blocco IF: f = i + 1
  B ENDIF                 ; salta blocco ELSE
ELSE    SUB R2, R2, R3    ; blocco ELSE: f = f - i
ENDIF
```

Istruzione `BGT` (Branch if Greater Than) - Selezione con "if-else-if"

```
; Codice eseguito se R0 è uguale a R1
CMP R0, R1        ; Confronto tra R0 e R1
BGT GreaterLabel  ; Salta a GreaterLabel se R0 è maggiore di
R1
BLT LessLabel     ; Salta a LessLabel se R0 è minore di R1

B   EndLabel      ; Salta a EndLabel

; Codice eseguito se R0 è maggiore di R1
GreaterLabel:
B   EndLabel

; Codice eseguito se R0 è minore di R1
LessLabel:

; Codice eseguito alla fine del costrutto di selezione
EndLabel:
```

Costrutto `switch/case`

```
; Supponiamo che la variabile di controllo sia nel registro R0

CMP R0, #1
BEQ Case1
CMP R0, #2
BEQ Case2
CMP R0, #3
BEQ Case3
; ... Altre casistiche ...
B DefaultCase

Case1:
; Codice eseguito per il caso 1
B EndSwitch

Case2:
; Codice eseguito per il caso 2
B EndSwitch

Case3:
; Codice eseguito per il caso 3
B EndSwitch

DefaultCase:
; Codice eseguito se nessuna delle casistiche precedenti è soddi

EndSwitch:
; Codice eseguito alla fine dello switch-case
```

ESEMPIO DI CODICE 6.15 **COSTRUTTO *SWITCH/CASE***

Codice di alto livello

```
switch (pulsante) {
  case 1:  bancomat = 20;  break;
  case 2:  bancomat = 50;  break;
  case 3:  bancomat = 100; break;
  default: bancomat = 0;
}
// codice equivalente con l'istruzione if/else
if      (pulsante == 1) bancomat = 20;
else if (pulsante == 2) bancomat = 50;
else if (pulsante == 3) bancomat = 100;
else                    bancomat = 0;
```

Codice *assembly* ARM

```
; R0 = pulsante, R1 = bancomat
  CMP R0, #1        ; è pulsante 1 ?
  MOVEQ R1, #20     ; bancomat = 20 se pulsante==1
  BEQ FINE          ; break
  CMP R0, #2        ; è pulsante 2 ?
  MOVEQ R1, #50     ; bancomat = 50 se pulsante==2
  BEQ FINE          ; break
  CMP R0, #3        ; è pulsante 3?
  MOVEQ R1, #100    ; bancomat = 100 se pulsante==3
  BEQ FINE          ; break
  MOV R1, #0        ; default bancomat = 0
FINE
```

Cicli

Ciclo `while`

```
LoopStart:
    ; Condizione di continuazione del ciclo
    CMP R0, #10         ; Supponiamo che R0 sia il contatore del
    BGE LoopEnd         ; Salta alla fine del ciclo se R0 è maggi

    ; Corpo del ciclo
    ; ... (codice del corpo del ciclo) ...

    ; Incremento del contatore
    ADD R0, R0, #1

    ; Salto all'inizio del ciclo
    B LoopStart

LoopEnd:
    ; Codice eseguito alla fine del ciclo
```

ESEMPIO DI CODICE 6.16 **CICLO *WHILE***

Codice di alto livello

```
int potenza = 1;
int x = 0;
while (potenza != 128) {
  potenza = potenza * 2;
  x = x + 1;
}
```

Codice *assembly* ARM

```
; R0 = potenza, R1 = x
        MOV R0, #1        ; potenza = 1
        MOV R1, #0        ; x = 0
WHILE   CMP R0, #128      ; potenza != 128 ?
        BEQ FINE          ; se potenza == 128, esce dal ciclo
        LSL R0, R0, #1    ; potenza = potenza * 2
        ADD R1, R1, #1    ; x = x + 1
        B WHILE           ; ripete il ciclo WHILE
FINE
```

Ciclo `for`

```
; Inizializzazione del contatore del ciclo
    MOV R0, #0

LoopStart:
    ; Condizione di continuazione del ciclo
    CMP R0, #10         ; Supponiamo che R0 sia il contatore del
    BGE LoopEnd         ; Salta alla fine del ciclo se R0 è maggi

    ; Corpo del ciclo
    ; ... (codice del corpo del ciclo) ...

    ; Incremento del contatore
    ADD R0, R0, #1

    ; Salto all'inizio del ciclo
    B LoopStart

LoopEnd:
    ; Codice eseguito alla fine del ciclo
```

ESEMPIO DI CODICE 6.18 **ACCESSO A UN ARRAY UTILIZZANDO UN CICLO *FOR***

Codice di alto livello

```
int i;
int punteggi[200];
...
for (i = 0; i < 200; i = i + 1)
  punteggi[i] = punteggi[i] + 10;
```

Codice *assembly* ARM

```
; R0 = indirizzo base dell'array, R1 = i
; inizializzazioni ...
        MOV R0, #0x14000000 ; R0 = indirizzo base
        MOV R1, #0 ; i = 0
FOR     CMP R1, #200        ; i < 200?
        BGE FINE            ; se i ≥ 200, esce dal ciclo
        LSL R2, R1, #2      ; R2 = i * 4
        LDR R3, [R0, R2]    ; R3 = punteggi[i]
        ADD R3, R3, #10     ; R3 = punteggi[i] + 10
        STR R3, [R0, R2]    ; punteggi[i] = punteggi[i] + 10
        ADD R1, R1, #1      ; i = i + 1
        B FOR               ; ripete il ciclo FOR
FINE
```

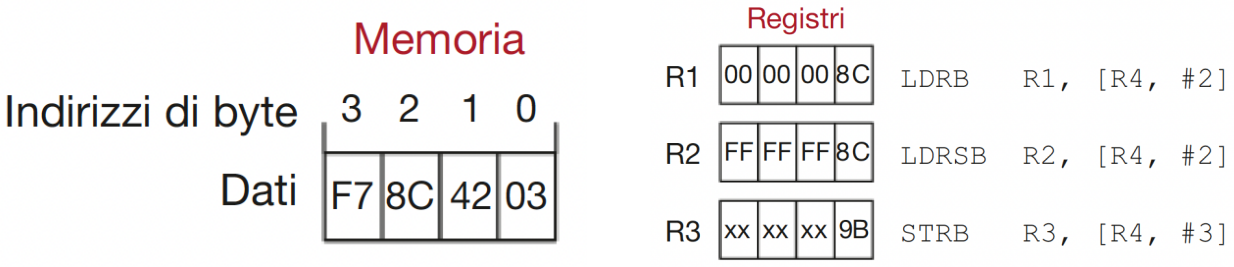
Esempio di implementazione di un array di parole:

```
.section .data
array:  .word  1, 2, 3, 4, 5   ; Array di 5 parole da 32 bit
```

- La direttiva `.section .data` è utilizzata per dichiarare una sezione di dati, e `array` è l'etichetta dell'array.
- La direttiva `.word` viene utilizzata per definire parole da 32 bit (o parole di 4 byte) con i valori 1, 2, 3, 4, e 5.

Tabella 6.5 Codifica ASCII.

#	Carat-tere	#	Carat-tere	#	Carat-tere	#	Carat-tere	#	Carat-tere	#	Carat-tere
20	spazio	30	0	40	@	50	P	60	`	70	p
21	!	31	1	41	A	51	Q	61	a	71	q
22	"	32	2	42	B	52	R	62	b	72	r
23	#	33	3	43	C	53	S	63	c	73	s
24	$	34	4	44	D	54	T	64	T	74	t
25	%	35	5	45	E	55	U	65	e	75	u
26	&	36	6	46	F	56	V	66	f	76	v
27	'	37	7	47	G	57	W	67	g	77	w
28	(	38	8	48	H	58	X	68	h	78	x
29	)	39	9	49	I	59	Y	69	i	79	y
2A	*	3A	:	4A	J	5A	Z	6A	j	7A	z
2B	+	3B	;	4B	K	5B	[	6B	k	7B	{
2C	,	3C	<	4C	L	5C	\	6C	l	7C	|
2D	-	3D	=	4D	M	5D	]	6D	m	7D	}
2E	.	3E	>	4E	N	5E	^	6E	n	7E	~
2F	/	3F	?	4F	O	5F	_	6F	o		



Funzioni

Le funzioni ricevono valori di ingresso, denominati parametri, e forniscono un solo valore di uscita, denominato valore di ritorno.

Le funzioni dovrebbero calcolare tale valore senza provocare altri effetti collaterali indesiderati.
Quando una funzione ne chiama un'altra, la prima funzione, detta il **chiamante**, e la seconda funzione, detta il **chiamato**, devono accordarsi su dove mettere i parametri e il valore di ritorno.

In **ARM**, convenzionalmente il chiamante mette fino a quattro parametri nei registri ***R0-R3*** prima di eseguire la chiamata a sottoprogramma, e il chiamato mette il valore di ritorno in ***R0*** prima di terminare.

> 💡 *Con questa convenzione entrambe le funzioni sanno dove trovare parametri*
>
> *e valore di ritorno, anche se sono state scritte da programmatori diversi*

Il chiamante memorizza l'indirizzo di ritorno nel registro **LR (Link Register)** nel momento in cui salta a eseguire il chiamato con l'istruzione **BL (Branch and Link)**.

Il chiamato *non* deve modificare nessun valore dell'architettura né della memoria dai quali dipende il successivo comportamento del chiamante.
In particolare deve lasciare inalterati i registri preservati (***R4-R11 oltre a LR***) e lo stack, ovvero la porzione di memoria usata dal chiamante per le proprie variabili temporanee.

Chiamata funzione

ARM usa l'istruzione BL (Branch and Link) per chiamare una funzione e copia il contenuto di tale registro nel **PC (MOV PC, LR)** per ritornare al chiamante.

ESEMPIO DI CODICE 6.20 **CHIAMATA DELLA FUNZIONE niente**

Codice di alto livello

```
int main() {
  niente();
  ...
}
// void = nessun valore di ritorno dalla funzione
void niente() {
  return;
}
```

Codice *assembly* ARM

```
0x00008000 MAIN    ...
...                ...
0x00008020         BL NIENTE    ; chiama la funzione "niente"
...
0x0000902C NIENTE  MOV PC, LR   ; ritorna
```

`BL` e `MOV PC`, `LR` sono le due istruzioni chiave per chiamata a e ritorno da funzione. BL fa due cose: memorizza l'indirizzo di ritorno (cioè l'indirizzo dell'istruzione successiva a BL) nel registro LR (Link Register) e salta alla destinazione.



Per convenzione ARM, le funzioni usano i registri R0-R3 per i parametri di ingresso e il registro R0 per il valore di ritorno.

ESEMPIO DI CODICE 6.21 **CHIAMATA DI FUNZIONE CON PARAMETRI E VALORE DI RITORNO**

Codice di alto livello

```
int main() {
  int y;
  . . .
  y = diffdisomme(2, 3, 4, 5);
  . . .
}
int diffdisomme(int f, int g, int h, int i) {
  int risultato;
  risultato = (f + g) - (h + i);
  return risultato;
}
```

Codice *assembly* ARM

```
; R4 = y
MAIN
   . . .
   MOV R0, #2        ; parametro 0 = 2
   MOV R1, #3        ; parametro 1 = 3
   MOV R2, #4        ; parametro 2 = 4
   MOV R3, #5        ; parametro 3 = 5
   BL DIFFDISOMME    ; chiamata della funzione
   MOV R4, R0        ; y = valore restituito
   . . .
; R4 = risultato
DIFFDISOMME
   ADD R8, R0, R1    ; R8 = f + g
   ADD R9, R2, R3    ; R9 = h + i
   SUB R4, R8, R9    ; risultato = (f + g) - (h + i)
   MOV R0, R4        ; mette il valore di ritorno in R0
   MOV PC, LR        ; ritorna al chiamante
```

Stack

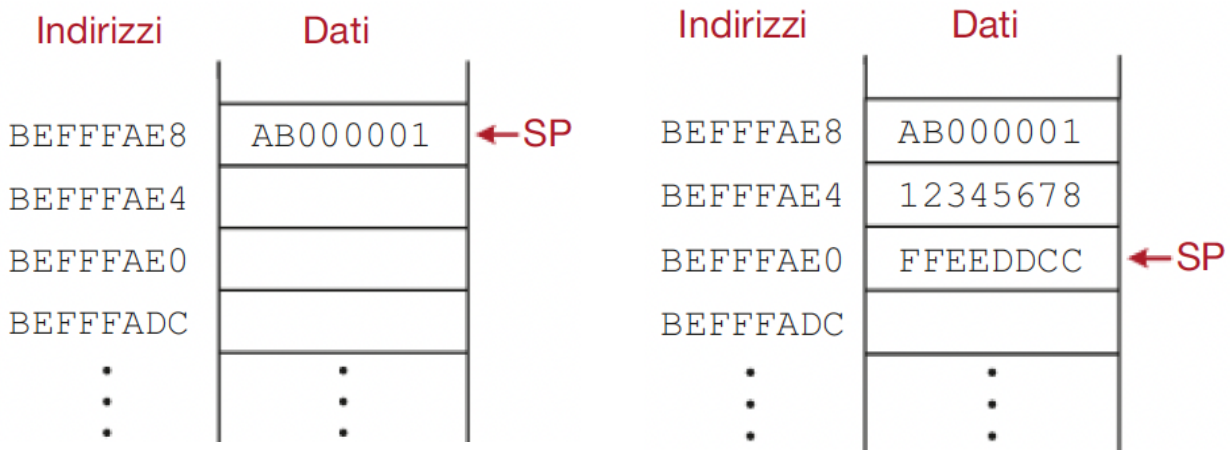
Lo stack che è una coda di tipo **FIFO (First In First Out)** è la parte di memoria usata per salvare informazioni all'interno di una funzione.

Si espande (cioè usa più memoria) quando il processore nell'esecuzione del programma ha bisogno di più spazio e si contrae (cioè libera memoria) quando il processore non ha più bisogno delle variabili temporanee memorizzate in precedenza.

Una funzione può allocare spazio sullo stack per memorizzare le variabili locali, ma deve deallocare tale spazio prima di ritornare al chiamante.

La cima dello stack è lo spazio allocato più di recente.

💡 Lo stack di ARM cresce in memoria verso il basso, cioè verso indirizzi minori quando il programma ha bisogno di spazio ulteriore.



📌 *Uno degli usi più importanti dello stack è il salvataggio e il ripristino dei*

registri usati da una funzione, che deve restituire il valore di ritorno senza altri

*effetti collaterali, in particolare senza modificare alcun registro tranne **R0**, che*

contiene il valore di ritorno.

Per evitare questo problema, la funzione deve salvare i registri sullo stack prima di modificarli e ripristinare i valori originari prima di terminare.

- In particolare, deve svolgere i seguenti passi.
- Salvare i valori dei registri nello stack.
- Eseguire le proprie attività utilizzando i registri salvati.
- Ripristinare i valori originali dei registri prelevandoli dallo stack.
- Deallocare lo spazio nello stack.
- Allocare spazio sullo stack per memorizzare i valori contenuti in uno o più registri.

ESEMPIO DI CODICE 6.22 **SALVATAGGIO E RIPRISTINO DI REGISTRI**

Codice *assembly* ARM

```
; R4 = risultato
DIFFDISOMME
   SUB SP, SP, #12   ; alloca spazio nello stack per 3 registri
   STR R9, [SP, #8]  ; salva R9 nello stack
   STR R8, [SP, #4]  ; salva R8 nello stack
   STR R4, [SP]      ; salva R4 nello stack
   ADD R8, R0, R1    ; R8 = f + g
   ADD R9, R2, R3    ; R9 = h + i
   SUB R4, R8, R9    ; risultato = (f + g) - (h + i)
   MOV R0, R4        ; mette il valore di ritorno in R0
   LDR R4, [SP]      ; recupera R4 dallo stack
   LDR R8, [SP, #4]  ; recupera R8 dallo stack
   LDR R9, [SP, #8]  ; recupera R9 dallo stack
   ADD SP, SP, #12   ; dealloca lo spazio nello stack
   MOV PC, LR        ; ritorna al chiamante
```

Salvataggio e ripristino di registri multipli

Il salvataggio e il ripristino dei registri sullo stack è un'operazione così frequente che ARM fornisce due istruzioni di accesso multiplo a memoria: **LDM (LoaD Multiple)** e **STM (STore Multiple)**, ottimizzate a tale scopo.

LDM e STM si presentano in quattro versioni:

- *Full Descending*
- *Empty Descending*
- *Full Ascending*
- *Empty Ascending*

Registri riservati

ESEMPIO DI CODICE 6.23 **SALVATAGGIO E RIPRISTINO DI REGISTRI MULTIPLI**

Codice *assembly* ARM

```
; R4 = risultato
DIFFDISOMME
   STMFD SP!, {R4, R8, R9} ; salva R4/8/9 in uno stack pieno discendente

   ADD R8, R0, R1          ; R8 = f + g
   ADD R9, R2, R3          ; R9 = h + i
   SUB R4, R8, R9          ; risultato = (f + g) - (h + i)
   MOV R0, R4              ; mette il valore di ritorno in R0

   LDMFD SP!, {R4, R8, R9} ; recupera R4/8/9 dallo stack pieno discendente

   MOV PC, LR              ; ritorna al chiamante
```

💡 *Ogni funzione deve quindi salvare e ripristinare tutti i registri preservati che intende usare, ma può modificare liberamente i registri non preservati.*

💡 I *registri preservati sono definiti "a salvataggio di chiamato" (**callee-save**), quelli non preservati "a salvataggio di chiamante" (**caller-save**).*

ESEMPIO DI CODICE 6.24 **RIDUZIONE DEL NUMERO DI REGISTRI PRESERVATI**

Codice *assembly* ARM

```
; R4 = risultato
DIFFDISOMME
   PUSH {R4}          ; salva R4 nello stack
   ADD R1, R0, R1     ; R1 = f + g
   ADD R3, R2, R3     ; R3 = h + i
   SUB R4, R1, R3     ; risultato = (f + g) - (h + i)
   MOV R0, R4         ; mette il valore di ritorno in R0
   POP {R4}           ; recupera R4 dallo stack
   MOV PC, LR         ; ritorna al chiamante
```

Le istruzioni `PUSH (e POP)` *salvano (e recuperano) i contenuti dei registri nello stack in ordine crescente di numero di registro, con il registro di numero minore nella cella di indirizzo più basso, indipendentemente dall'ordine indicato nell'istruzione assembly*

ESEMPIO DI CODICE 6.24 **RIDUZIONE DEL NUMERO DI REGISTRI PRESERVATI**

Codice *assembly* ARM

```
; R4 = risultato
DIFFDISOMME
   PUSH {R4}          ; salva R4 nello stack
   ADD R1, R0, R1     ; R1 = f + g
   ADD R3, R2, R3     ; R3 = h + i
   SUB R4, R1, R3     ; risultato = (f + g) - (h + i)
   MOV R0, R4         ; mette il valore di ritorno in R0
   POP {R4}           ; recupera R4 dallo stack
   MOV PC, LR         ; ritorna al chiamante
```

Funzione che non chiama un altra funzione

Questa tipologia di funzione è un po' più complicata perché può dover salvare sullo stack alcuni registri non preservati prima di chiamare a sua volta una

funzione, e ripristinarli successivamente.
In particolare:

- regola di salvataggio del chiamante: prima di una chiamata a funzione, il chiamante deve salvare ogni registro non preservato (**R0–R3 e R12**) che intende usare dopo la chiamata, e successivamente ripristinarlo prima di utilizzarlo
- regola di salvataggio del chiamato: prima di modificare un registro preservato (**R4–R11 e LR**), il chiamato deve salvarne il contenuto e ripristinarlo prima di ritornare al chiamante.

ESEMPIO DI CODICE 6.26 **CHIAMATA DI FUNZIONE NON-FOGLIA**

Codice di alto livello

```
int f1(int a, int b) {
  int i, x;
  x = (a + b)*(a - b);
  for (i=0; i<a; i++)
    x = x + f2(b+i);
  return x;
}

int f2(int p) {
  int r;
  r = p + 5;
  return r + p;
}
```

Codice *assembly* ARM

```
; R0 = a, R1 = b, R4 = i, R5 = x
F1
  PUSH {R4, R5, LR ; salva i registri preservati usati da f1
  ADD R5, R0, R1   ; x = (a + b)
  SUB R12, R0, R1  ; temp = (a - b)
  MUL R5, R5, R12  ; x = x * temp = (a + b) * (a - b)
  MOV R4, #0       ; i = 0
CICLO
  CMP R4, R0       ; i < a?
  BGE RITORNO      ; no: esce dal ciclo
  PUSH {R0, R1}    ; salva i registri non preservati
  ADD R0, R1, R4   ; il parametro è b + i
  BL F2            ; chiama f2(b+i)
  ADD R5, R5, R0   ; x = x + f2(b+i)
  POP {R0, R1}     ; ripristina i registri non preservati
  ADD R4, R4, #1   ; i++
  B CICLO          ; ripete il ciclo for
RITORNO
  MOV R0, R5 ; il valore di ritorno è x
  POP {R4, R5, LR} ; ripristina i registri preservati
  MOV PC, LR       ; ritorno da f1

; R0 = p, R4 = r
F2
  PUSH {R4}        ; salva i registri preservati usati da f2
  ADD R4, R0, 5    ; r = p + 5
  ADD R0, R4, R0   ; il valore di ritorno è r + p
  POP {R4}         ; ripristina i registri preservati
  MOV PC, LR       ; ritorno da f2
```

Chiamata di funzioni ricorsive

Una funzione ricorsiva è una funzione non-foglia che chiama se stessa.
Si comporta quindi sia da chiamante sia da chiamato, e deve pertanto salvare sia i registri preservati sia quelli non preservati.

ESEMPIO DI CODICE 6.27 **LA FUNZIONE RICORSIVA `fattoriale`**

Codice di alto livello

```
int fattoriale(int n) {
  if (n <= 1)
    return 1;




  else
    return (n * fattoriale(n - 1));
}
```

Codice *assembly* ARM

```
0x8500  FATTORIALE  PUSH {R0, LR}      ; salva n e LR sullo stack
0x8504              CMP R0, #1         ; R0 <= 1?
0x8508              BGT ELSE           ; no: salta al ramo else
0x850C              MOV R0, #1         ; altrimenti restituisce 1
0x8510              ADD SP, SP, #8     ; ripristina SP
0x8514              MOV PC, LR         ; ritorna
0x8518  ELSE        SUB R0, R0, #1     ; n = n - 1
0x851C              BL FATTORIALE      ; chiamata ricorsiva
0x8520              POP {R1, LR}       ; ripristina n (in R1) e LR
0x8524              MUL R0, R1, R0     ; R0 = n * fattoriale(n - 1)
0x8528              MOV PC, LR         ; ritorna
```

Secondo la regola di salvataggio del chiamato, fattoriale è una funzione non-foglia e deve salvare `LR`. Secondo la regola di salvataggio del chiamante, fattoriale avrà bisogno di n dopo che si sarà chiamata, quindi deve salvare `R0`.

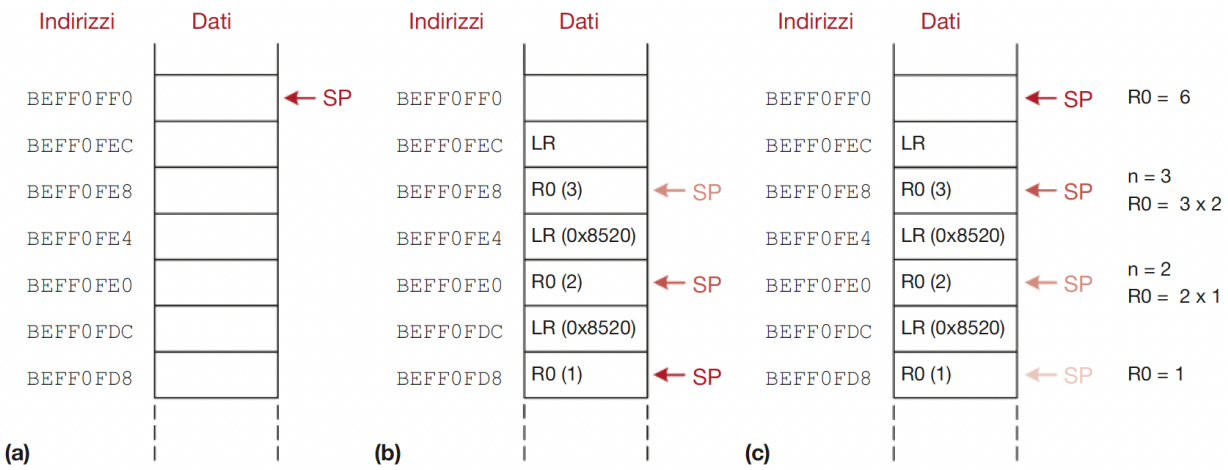
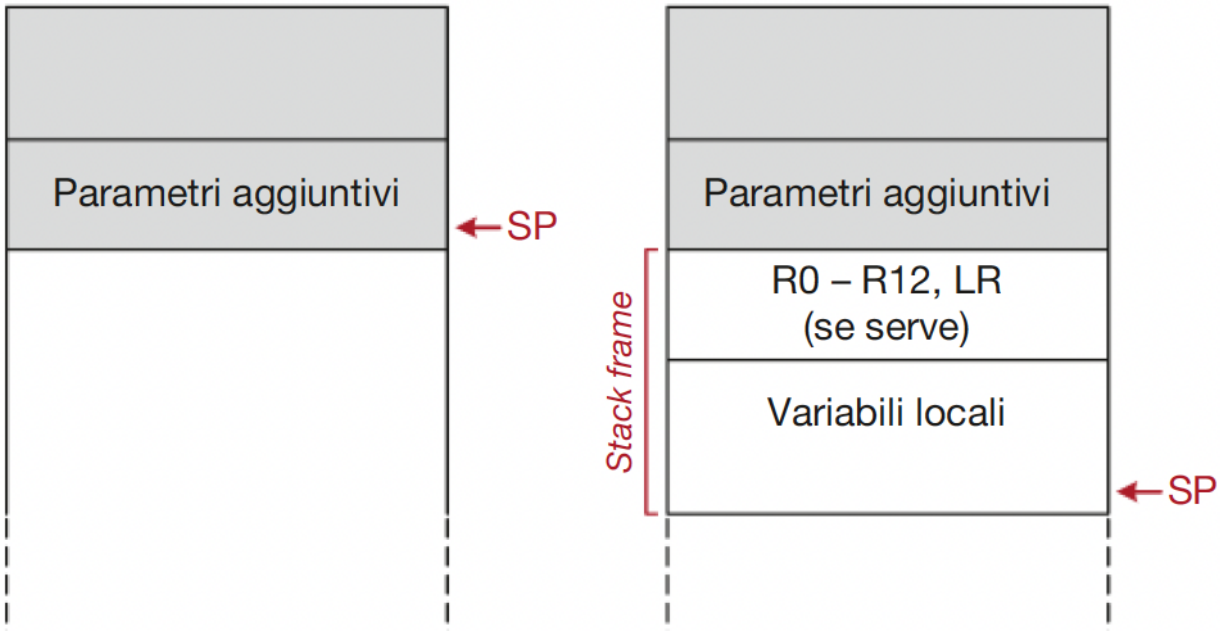


Figura 6.14 **Lo stack: (a) prima, (b) durante e (c) dopo la chiamata alla funzione fattoriale con `n` = 3.**

Parametri aggiuntivi e variabili locali

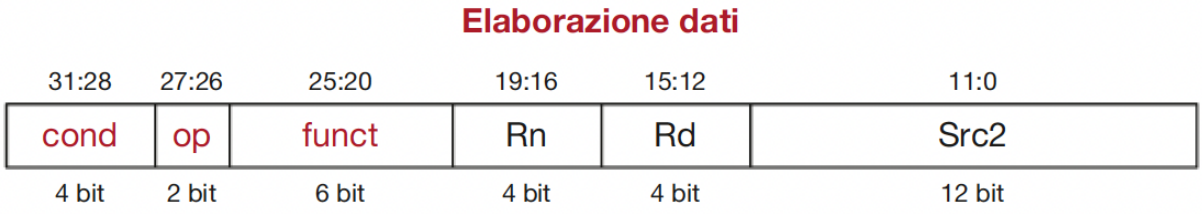
Le funzioni possono avere più di quattro parametri e troppe variabili locali per poterle memorizzare tutte nei registri preservati. Per memorizzare queste informazioni si usa lo stack.

Per convenzione ARM, se una funzione ha più di quattro parametri, i primi quattro sono passati nei registri di parametro come al solito; i parametri aggiuntivi sono memorizzati sullo stack, appena sopra `SP`.



Istruzioni di elaborazione dati

Il *primo* operando sorgente è un ***registro***, il *secondo* può essere un ***immediato*** oppure un ***registro***, eventualmente traslato. Un *terzo* registro è la ***destinazione***.



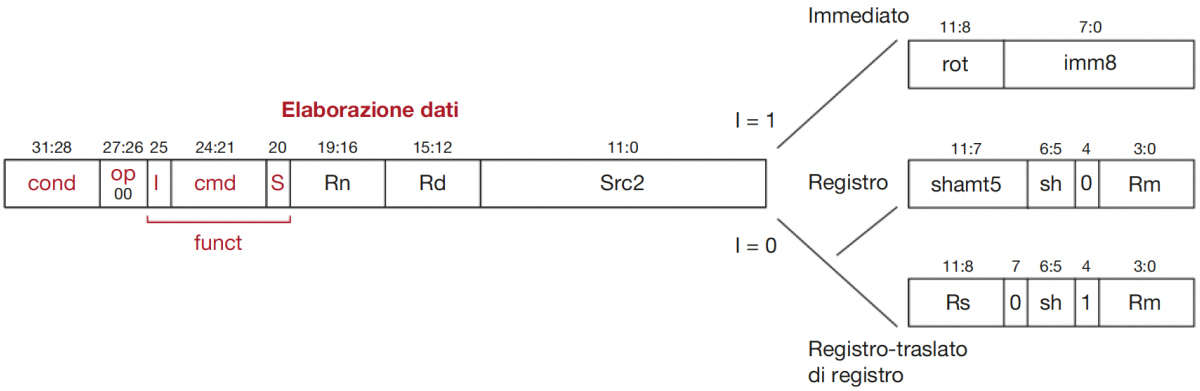
> 💡 *L'istruzione a **32 bit** ha sei campi:* `cond`, `op`, `funct`, `Rn`, `Rd` *e* `Src2`

Gli operandi sono codificati in tre campi

1. `Rn` è il registro del primo operando sorgente
2. `Src2` è il secondo operando sorgente
3. `Rd` è il registro destinazione

Le istruzioni di elaborazione dati hanno anche una rappresentazione inconsueta basata su un immediato a ***8 bit***, ***imm8***, e su una rotazione a ***4 bit***, rot.
imm8 viene ruotato a destra di 2 x rot per costruire una costante a 32 bit.

> 💡 *`Rd` è l'abbreviazione di "registro destinazione".*
>
> *`Rn` e `Rm` indicano il primo e il secondo registro sorgente.*

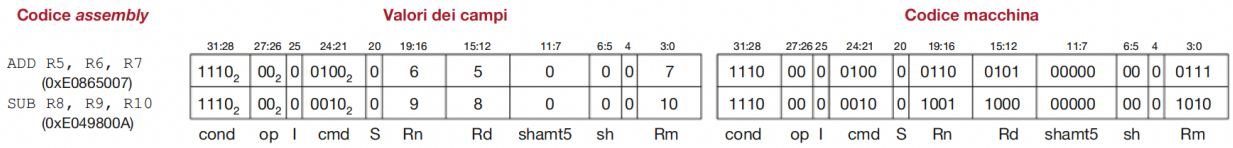


Formato delle istruzioni di elaborazione dati con dettagli del campo funct e delle alternative per Src2.

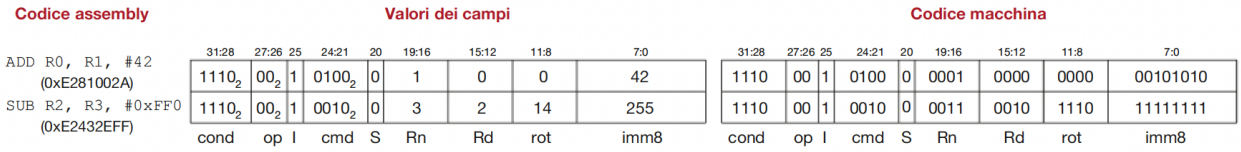
Tabella 6.7 **Rotazioni immediate e costanti a 32 bit risultanti per *imm8* = 0xFF.**

rot	costante a 32 bit							
0000	0000	0000	0000	0000	0000	0000	1111	1111
0001	1100	0000	0000	0000	0000	0000	0011	1111
0010	1111	0000	0000	0000	0000	0000	0000	1111
...				...				
1111	0000	0000	0000	0000	0000	0011	1111	1100

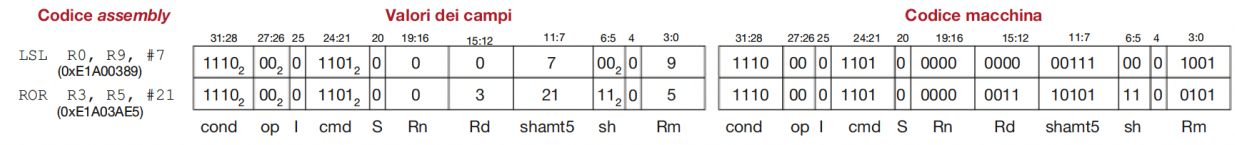
Rotazioni immediate e costanti a 32 bit risultanti per imm8 = 0xFF



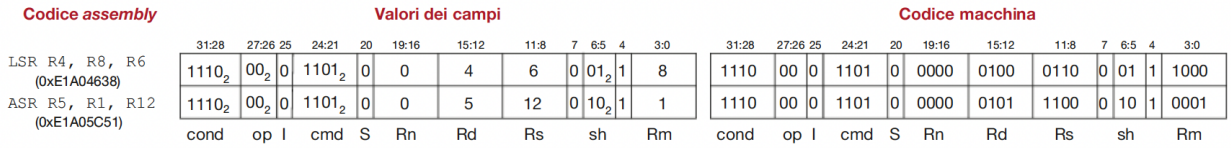
*codice macchina per le **istruzioni** `ADD` e `SUB` con un operando immediato e due operandi a registro*



*codice macchina per la **traslazione logica a sinistra** (`LSL`) e per la **rotazione a destra** (`ROT`) con valori di traslazione immediati*



`Rm` *(cioè R5) contiene il valore a 32 bit da traslare, e* `shamt5` *specifica il valore di traslazione.*
Il valore traslato viene memorizzato in `Rd`*. Rn non è usato, quindi viene lasciato a 0*



codice macchina per `LSR` e `ASR` con il valore di traslazione codificato negli 8 bit meno significativi di Rs (rispettivamente R6 e R12)

Questa istruzione usa il modo di indirizzamento a registro con traslazione a registro, nel quale un registro (Rm) viene traslato del valore contenuto in un altro registro (Rs). Dal momento che si usano gli 8 bit meno significativi di Rs, Rm può essere traslato al massimo di 255 posizioni.

Istruzione	sh	Operazione
LSL	00_2	Logical shift left
LSR	01_2	Logical shift right
ASR	10_2	Arithmetic shift right
ROR	11_2	Rotate right

Codifiche del campo sh

P	W	Modo di gestione indice
0	0	Post-indice
0	1	Non supportato
1	0	Spiazzamento
1	1	Pre-indice

Bit di controllo del modo di gestione indice per istruzioni di accesso a memoria

L	B	Istruzione
0	0	STR
0	1	STRB
1	0	LDR
1	1	LDRB

Bit di controllo del tipo di operazione per istruzioni di accesso a memoria

Istruzioni di accesso a memoria

Le istruzioni di accesso a memoria usano un formato simile a quello delle istruzioni di elaborazione dati, con gli stessi sei campi `cond`, `op`, `funct`, `Rn`, `Rd` e `Src2`

Hanno però una diversa codifica del campo `funct`, due varianti del campo Src2 e il campo

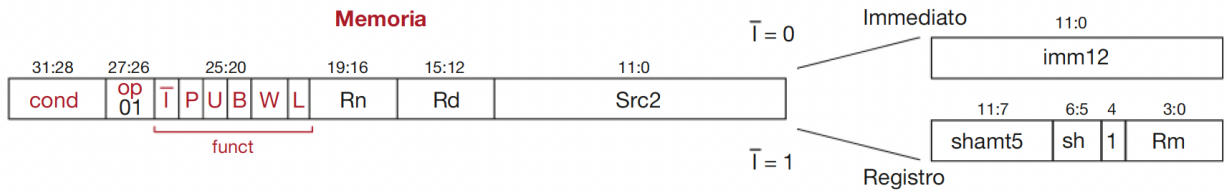
`op = 012`.

`Rn` è il ***registro base***, `Src2` costituisce lo ***spiazzamento*** (offset) e `Rd` è il ***registro destinazione*** nelle istruzioni di lettura da memoria e il registro sorgente in quelle di scrittura in memoria. L'offset può essere un immediato unsigned a 12 bit (`imm12`) oppure un registro (`Rm`) eventualmente traslato di una costante (shamt5).

`funct` è costituito da sei bit di controllo: **$\bar{I}$**, **P**, **U**, **B**, **W** e **L**. I due bit **$\bar{I}$** (**immediate**) e **U** (**add**) determinano se lo spiazzamento è un immediato o un registro

Bit	Significato	
	$\bar{I}$	U
0	Spiazzamento immediato in Src2	Sottrae lo spiazzamento dalla base
1	Spiazzamento a registro in Src2	Somma lo spiazzamento alla base

Bit di controllo del tipo di spiazzamento per istruzioni di accesso a memoria.



*Formato delle istruzioni di accesso a memoria **LDR**, **STR**, **LDRB** e **STRB***

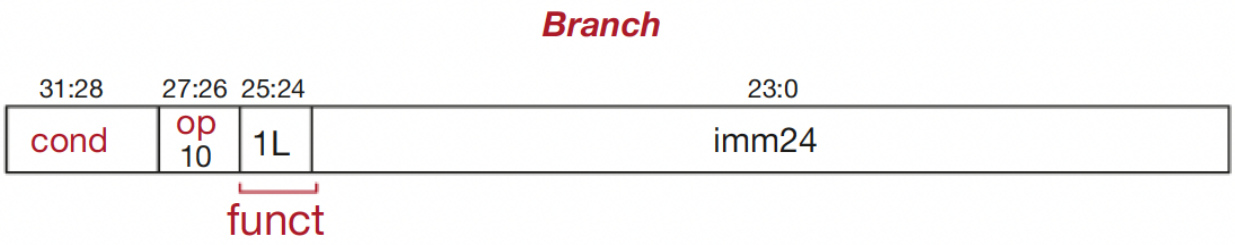
Istruzioni di salto

Le istruzioni di salto usano un solo operando immediato signed a 24 bit, `imm24`

Come le istruzioni di elaborazione dati e di accesso a memoria, anche i salti cominciano con un campo a 4 bit per l'esecuzione condizionata e un campo $op = 10_2$.

Il campo `funct` è di soli 2 bit: il più significativo è sempre 1, il meno significativo, **L**, indica il tipo di salto: 1 per `BL`, 0 per `B`

Il rimanente campo `imm24` codificato come numero in complemento a due di 24 bit serve a specificare l'indirizzo dell'istruzione alla quale saltare, relativamente a *PC + 8*.

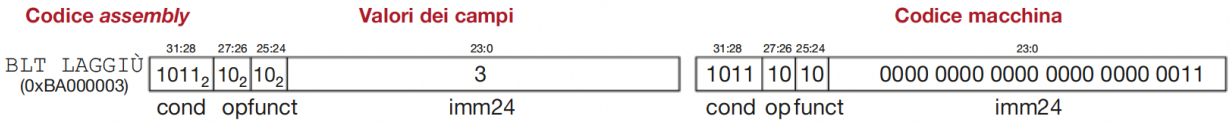


Formato delle istruzioni di salto

ESEMPIO DI CODICE 6.28 **CALCOLO DELL'INDIRIZZO DI DESTINAZIONE DEL SALTO**

Codice *assembly* ARM

```
0x80A0          BLT LAGGIU
0x80A4          ADD R0, R1, R2
0x80A8          SUB R0, R0, R9
0x80AC          ADD SP, SP, #8
0x80B0          MOV PC, LR
0x80B4  LAGGIU  SUB R0, R0, #1
0x80B8          ADD R3, R3, #0x5
```



*Codice macchina dell'istruzione di salto se minore di (**BLT**)*

Il campo immediato a 24 bit (`imm24`) contiene il numero di istruzioni comprese tra *BTA e PC + 8* (ovvero due istruzioni dopo quella di salto). In questo caso, il valore di `imm24` nell'istruzione `BLT` vale 3 perché `BTA` (***0x80B4***) è appunto tre istruzioni dopo *PC + 8* (***0x80A8***).

Il processore calcola `BTA` a partire dall'istruzione estendendo a 32 bit il segno dell'immediato a 24 bit, traslandolo a sinistra di 2 posizioni (per convertire indirizzi di parola in indirizzi di byte) e sommandolo a *PC + 8*.

Modi di indirizzamento

ARM utilizza quattro modi principali: a registro, immediato, base e relativo al PC

Gli indirizzamenti a registro e base hanno molti sotto-modi descritti oltre. I primi tre modi
(
a registro, immediato e base) definiscono dove leggere e scrivere gli operandi, l'ultimo modo relativo al PC definisce come modificare appunto il `PC` (*Program Counter*).

Modo di indirizzamento dell'operando	Esempio	Descrizione
A registro		
A registro semplice	`ADD R3, R2, R1`	`R3 ← R2 + R1`
A registro con traslazione immediata	`SUB R4, R5, R9, LSR #2`	`R4 ← R5 - (R9 >> 2)`
A registro con traslazione a registro	`ORR R0, R10, R2, ROR R7`	`R0 ← R10 | (R2 ROR R7)`
Immediato	`SUB R3, R2, #25`	`R3 ← R2 - 25`
Base		
Spiazzamento immediato	`STR R6, [R11, #77]`	`mem[R11 + 77] ← R6`
Spiazzamento a registro	`LDR R12, [R1, -R5]`	`R12 ← mem[R1 - R5]`
Spiazzamento a registro con traslazione immediata	`LDR R8, [R9, R2, LSL #2]`	`R8 ← mem[R9 + (R2 << 2)]`
Relativo al PC	`B DEST`	`Salta a DEST`

Le istruzioni di elaborazione dati usano indirizzamento a registro o immediato, dove il primo operando sorgente è un registro e il secondo un registro o un immediato.

ARM consente che il secondo registro possa essere traslato di un numero di posizioni specificato in un immediato oppure in un terzo registro. Le istruzioni di accesso a memoria usano indirizzamento base, nel quale l'indirizzo base viene da un registro e lo spiazzamento (offset) da un immediato, da un registro o da un registro traslato di un immediato.

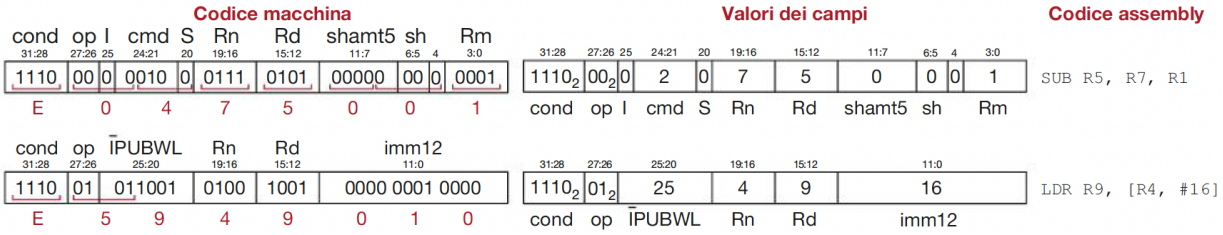
I salti usano indirizzamento relativo al `PC`, con l'indirizzo di destinazione del salto calcolato aggiungendo uno spiazzamento a *PC + 8*.

ESEMPIO

Tradurre dal linguaggio macchina al linguaggio assembly.
Tradurre le seguenti istruzioni macchina in istruzioni assembly.

- 0xE0475001
- 0xE5949010

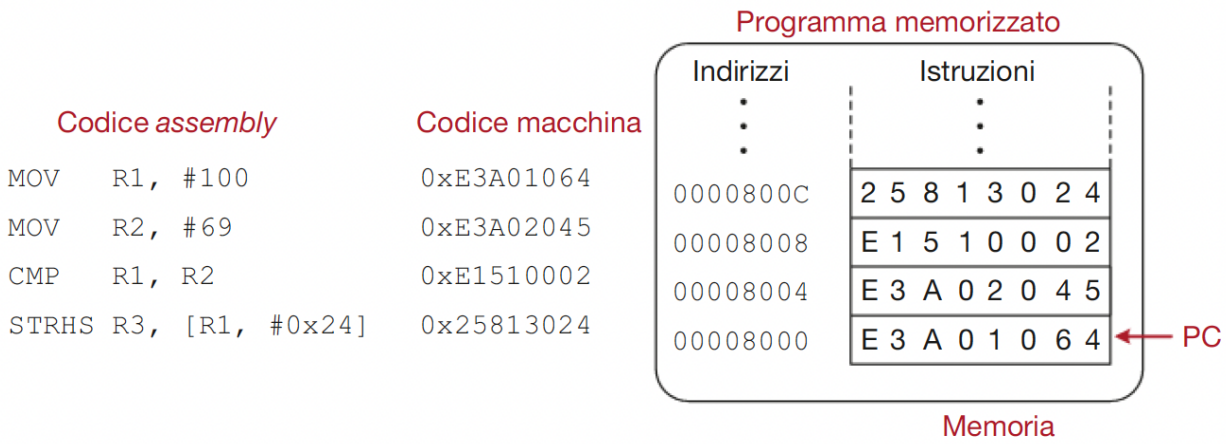
SOLUZIONE:



Scrivere un programma in memoria

Un programma scritto in linguaggio macchina è una serie di numeri a 32 bit che rappresentano le istruzioni, con ciò Invece di richiedere un hardware dedicato, si può lavorare con hardware di tipo generale (***general purpose***) ed eseguire applicazioni diverse (calcoli numerici, elaborazione di testi, visualizzazione di filmati) semplicemente cambiando il programma in memoria.

*Le istruzioni del programma vengono recuperate (**fetched**) dalla memoria* ed eseguite dal processore: quindi programmi anche complessi sono in ultima analisi una serie di letture da memoria ed esecuzioni di istruzioni.



Come sono memorizzate in memoria le istruzioni

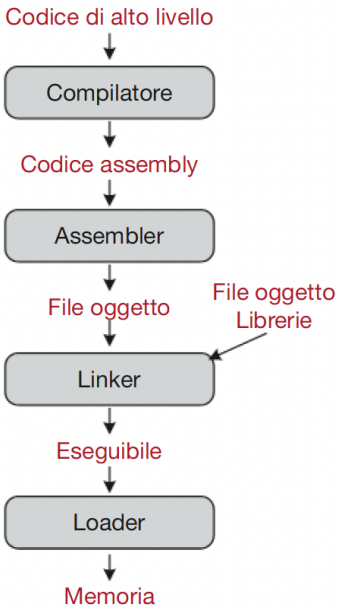
In ARM si parte normalmente da indirizzi bassi (in questo caso 0x00008000): si deve ricordare che la memoria di ARM è indirizzata a byte, quindi gli indirizzi delle istruzioni (di 32 bit, quindi di 4 byte) avanzano di 4 in 4.

Per eseguire il programma memorizzato (***run***) il processore esegue il fetch delle istruzioni in sequenza. Dopo il ***fetch*** l'istruzione viene decodificata ed eseguita dall'hardware del processore.

L'indirizzo dell'istruzione corrente è mantenuto nel registro denominato *program counter* (`PC`), che in ARM è `R15`. Per ragioni storiche, un'operazione di lettura del registro PC restituisce l'indirizzo dell'istruzione corrente più 8.

In ARM tale stato comprende il contenuto dei registri di lavoro e dei registri di stato.
Se il sistema operativo salva lo stato di esecuzione di un programma a un certo punto, può interrompere il programma, fare altro, poi ripristinare lo stato del programma interrotto e riprendere l'esecuzione senza che il programma stesso abbia risentito dell'interruzione.

Compilare, assemblare e caricare



Passi per tradurre e lanciare un programma

Per prima cosa il compilatore traduce il codice di alto livello in codice assembly.

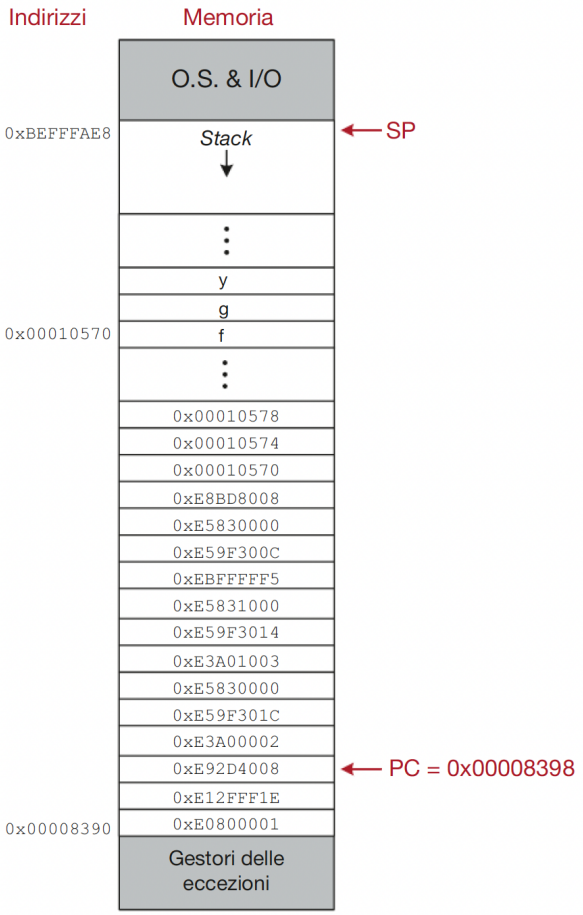
Quindi l'assembler traduce il codice assembly in codice macchina e lo salva nel file oggetto (object). Successivamente il linker unisce il codice macchina con quello delle librerie e di altri file, determina i corretti indirizzi di salto e le locazioni delle variabili, e produce l'intero programma eseguibile.

In pratica, la maggior parte dei compilatori esegue tutte le tre fasi di compilazione, assemblaggio e collegamento.
Da ultimo, il loader carica il programma in memoria e ne inizia l'esecuzione.

La mappa di memoria

Con indirizzi a 32 bit, ARM ha uno spazio di indirizzamento di 232 byte (4 GB). Gli indirizzi di parola sono multipli di 4, e vanno quindi da 0 a 0xFFFFFFFC.

L'architettura ARM divide lo spazio di indirizzamento in cinque parti o segmenti: il segmento testo, il segmento dati globali, il segmento dati dinamici, il segmento per i gestori delle eccezioni, il segmento per il sistema operativo (OS) e per l'ingresso/ uscita (I/O). Di seguito è descritto ciascuno di questi segmenti.



Il segmento di testo

Il segmento testo contiene il programma in linguaggio macchina. ARM chiama questo segmento anche segmento a sola lettura (`RO`, **Read Only**).
Oltre al codice può contenere costanti (dette anche literal) e dati a sola lettura.

Il segmento dati globali

Il segmento dati globali memorizza le variabili globali che, al contrario di quelle locali, sono accessibili a tutte le funzioni del programma.

Le variabili globali vengono allocate in memoria prima di iniziare l'esecuzione del programma. ARM chiama questo segmento anche segmento a lettura/scrittura (`RW`, ***Read Write***).
Si accede alle variabili globali tipicamente con un registro base statico che punta all'inizio del segmento dati globali. ARM usa per convenzione `R9` a questo scopo (`SB`, ***Static Base register***).

Il segmento dati dinamici
Il segmento dati dinamici contiene lo stack e l'heap.
All'inizio, il sistema operativo carica nello stack pointer (`SP`) l'indirizzo della cima del segmento, perché di solito SP cresce verso il basso, come mostrato in questo testo.
Lo stack contiene i dati temporanei e le variabili locali, come gli array, che sono troppo grandi per stare nei registri.
Al contrario di quanto avviene per lo stack, i dati nell'heap possono essere utilizzati ed eliminati in qualsiasi ordine. L'heap normalmente cresce verso l'alto, dall'inizio del segmento dati dinamici.

I segmenti dei gestori delle eccezioni, di OS e di I/O
La parte più bassa della memoria di ARM è riservata alla tabella del vettore delle eccezioni e ai gestori delle eccezioni, a partire dall'indirizzo 0x0. La parte più alta della memoria è riservata al sistema operativo e all'ingresso/ uscita mappato in memoria

Compilazione

Per compilare, assemblare e collegare un programma in C di nome `prog.c` si deve usare il comando:

```
gcc –O1 –g prog.c –o prog
```

Questo comando produce un codice eseguibile denominato prog.
L'opzione-O1 dice al compilatore di effettuare le ottimizzazioni essenziali invece di

produrre codice grossolanamente inefficiente. L'opzione -g dice al compilatore di includere informazioni utili al collaudo (debugging) del programma.

Per vedere i passi intermedi si può usare l'opzione -S che dice a GCC di compilare senza assemblare né collegare.

```
gcc –O1 –S prog.c –o prog.s
```

ESEMPIO DI CODICE 6.29 **COMPILAZIONE DI UN PROGRAMMA DI ALTO LIVELLO**

Codice di alto livello

```
int f, g, y; // variabili globali


int somma(int a, int b) {
  return (a + b);
}


int main(void)
{
  f = 2;
  g = 3;
  y = somma(f, g);
return y;
}
```

Codice *assembly* ARM

```
  .text
  .global  somma
  .type    somma, %function
somma:
  add      r0, r0, r1
  bx       lr
  .global  main
  .type    main, %function
main:
  push     {r3, lr}
  mov      r0, #2
  ldr      r3, .L3
  str      r0, [r3, #0]
  mov      r1, #3
  ldr      r3, .L3+4
  str      r1, [r3, #0]
  bl       somma
  ldr      r3, .L3+8
  str      r0, [r3, #0]
  pop      {r3, pc}
.L3:
  .word    f
  .word    g
```

Assemblaggio

L'assembler converte il linguaggio assembly in un file oggetto (object) contenente codice macchina. GCC può creare il file oggetto sia da prog.s sia direttamente da prog.c usando:

```
gcc –c prog.s –o prog.o
```

oppure

```
gcc –O1 –g –c prog.c –o prog.o
```

L'assembler fa *due passate* sul codice assembly.
Nella
prima passata assegna gli indirizzi alle istruzioni e individua tutti i simboli, cioè le label e i nomi delle variabili globali; nomi e indirizzi dei simboli sono salvati nella tabella dei simboli.

Nella ***seconda passata*** produce il codice macchina prelevando gli indirizzi delle label e delle variabili globali dalla tabella dei simboli.
Il codice macchina e la tabella dei simboli sono salvati nel file oggetto.

Si può "disassemblare" il file oggetto con il comando objdump per vedere il codice assembly corrispondente al codice macchina generato.
Se il codice di alto livello era stato compilato con l'opzione
`-g`, il disassembler mostra anche
le corrispondenti righe di codice C:

```
objdump -S prog.o
```

Ecco il disassemblato della sezione `.text`

```
00000000 <somma>:
int somma(int a, int b) {
    return (a + b);
}
```

```
    0: e0800001 add r0, r0, r1
    4: e12fff1e bx lr
00000008 <main>:

int f, g, y; // variabili globali

int somma(int a, int b);

int main(void) {
8: e92d4008 push {r3, lr}
    f = 2;
c: e3a00002 mov r0, #2
10: e59f301c ldr r3, [pc, #28] ; 34 <main+0x2c>
14: e5830000 str r0, [r3]
    g = 3;
18: e3a01003 mov r1, #3
1c: e59f3014 ldr r3, [pc, #20] ; 38 <main+0x30>
20: e5831000 str r1, [r3]
    y = somma(f,g);
24: ebffffe bl 0 <somma>
28: e59f300c ldr r3, [pc, #12] ; 3c <main+0x34>
2c: e5830000 str r0, [r3]
return y;
}

30: e8bd8008 pop {r3, pc}
...
```

Si può anche vedere la tabella dei simboli del file oggetto con l'opzione -t del comando objdump.

```
objdump –t prog.o
```

```
SYMBOL TABLE:
00000000 l d .text 00000000 .text
00000000 l d .data 00000000 .data
00000000 g F .text 00000008 somma
00000008 g F .text 00000038 main
00000004 O *COM* 00000004 f
00000004 O *COM* 00000004 g
00000004 O *COM* 00000004 y
```

Collegamento

Il compito del linker è quello di unire tutti i file oggetto e il codice di inizializzazione in un unico file in linguaggio macchina definito eseguibile (***executable***) e assegnare gli indirizzi alle variabili globali.

Il ***linker*** riloca dati e istruzioni dei file oggetto in modo che non si sovrappongano e usa le tabelle dei simboli per risistemare il codice macchina sulla base dei nuovi indirizzi delle label e delle variabili globali. Per collegare il file oggetto si attiva il linker con il comando:

```
gcc prog.o –o prog
```

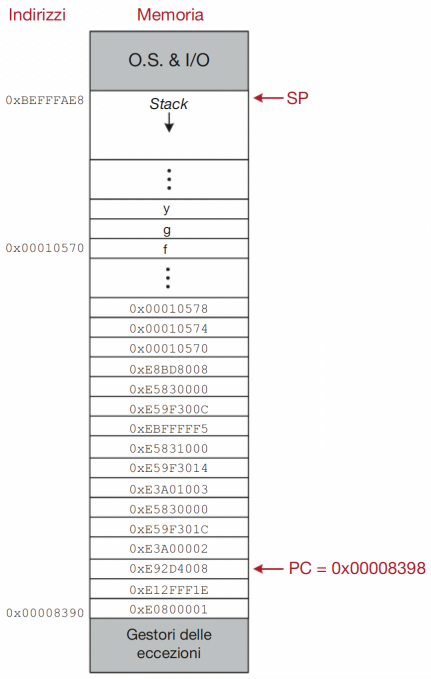
Si può disassemblare nuovamente l'eseguibile con il comando:

```
objdump -S -t prog
```

(L'eseguibile contiene anche una tabella dei simboli aggiornata con gli indirizzi rilocati delle funzioni e delle variabili globali.)

Caricamento

Il sistema operativo carica in memoria un programma copiando il segmento testo del file eseguibile da un dispositivo di memoria di massa (solitamente il disco rigido della macchina) nel segmento testo della memoria di lavoro.
Poi esegue un salto alla prima istruzione del programma per iniziarne l'esecuzione



Caricamento di literal

L'istruzione MOV prevede una sorgente a 12 bit, quindi serve usare l'istruzione LDR per caricare questi valori prelevandoli da un serbatoio di literal nel segmento testo. ARM consente questi caricamenti nelle forme:

```
LDR Rd, =literal
LDR Rd, =label
```

La prima forma carica nel registro Rd la costante a 32 bit specificata da **literal**, mentre la seconda carica l'indirizzo di memoria associato a **label**.

In entrambi i casi il valore viene mantenuto nel serbatoio dei literal, una parte del segmento testo. Tale serbatoio deve trovarsi a meno di 4096 byte di distanza dall'istruzione LDR in modo che il caricamento si possa fare con LDR Rd, [PC, #offset_dal_literal]

ESEMPIO DI CODICE 6.30	CAMPO IMMEDIATO DI GRANDI DIMENSIONI USANDO UN SERBATOIO DEI LITERAL
Codice di alto livello	**Codice *assembly* ARM**

`int a = 0x2B9056F;`	`; R1 = a`
	`LDR R1, =0x2B9056F`
	`...`

NOP

`NOP` è lo mnemonico di NO Operation. Si tratta di una pseudo-istruzione che non fa nulla: l'assembler la traduce come `MOV R0, R0` (in codice macchina 0xE1A00000). Tra i suoi utilizzi vi sono introdurre ritardi e allineare istruzioni.